



ANTIVIRUS INTEGRATION

Operating System



FEBRUARY 12, 2026

PROJECT BY
AbdulRahman Awan

1. Introduction

With the increasing number of malware attacks and malicious software, operating systems require protection mechanisms to ensure system security and stability. Antivirus software plays an essential role in detecting and preventing harmful files from executing on a system.

This project aims to develop a simplified Antivirus Integration System using C++ that demonstrates how antivirus mechanisms interact with core operating system components such as the file system and process management.

2. Problem Statement

Operating systems are vulnerable to malicious files that can compromise data integrity and system performance. Many systems lack basic monitoring tools to detect infected files.

The problem is to design a system that:

- Scans files for malicious signatures
- Detects infected files
- Prevents further execution
- Demonstrates integration with OS-level file handling

3. Objectives

The main objectives of this project are:

- To implement a file scanning mechanism using C++
- To generate hash values for file verification
- To compare file hashes with a stored virus signature database
- To implement a quarantine system for infected files
- To demonstrate operating system concepts such as file handling and directory traversal

4. Proposed System Overview

The Antivirus Integration System will consist of the following modules:

◆ 1. File Scanning Module

- Traverse selected directories
- Read files in binary mode
- Generate a custom hash value
- Compare with virus signature database

◆ 2. Signature Database

- Stored in a simple text file (virus.txt)
- Contains hash values of known infected files
- Easily editable and updatable

◆ 3. Quarantine Module

- Automatically moves infected files to a secure "quarantine" folder
- Prevents further execution

◆ 4. User Interface

- Console-based interface
- Allows user to:
 - Select directory
 - Start scan
 - View results

5. Technologies and Tools

Programming Language:

- C++

Development Environment:

- Visual Studio (Community Edition)

Libraries Used:

- <iostream> – Input/output
- <fstream> – File handling
- <filesystem> – Directory traversal
- <string> – String operations
- <vector> – Data storage

No external libraries or advanced frameworks will be used.

6. Methodology

The project will be developed using the following steps:

1. Requirement analysis
2. System design and architecture planning
3. Implementation of file scanning functionality
4. Implementation of hash generation algorithm
5. Integration with signature database
6. Implementation of quarantine system
7. Testing with sample files
8. Documentation and final presentation

7. System Architecture

User

↓

Directory Selection

Antivirus Integration

↓

File Scanner (C++ Engine)

↓

Hash Generation

↓

Signature Database Comparison

↓

If Match → Move to Quarantine

Else → Mark as Safe

8. Expected Outcomes

- A working prototype of antivirus integration system
- Ability to detect predefined malicious files
- Automatic quarantine of infected files
- Practical understanding of file system interaction in OS

9. Project Scope

Included:

- Directory scanning
- File reading and hashing
- Signature comparison
- Quarantine functionality
- Console-based interface

Not Included:

- Real-time kernel monitoring
- Advanced malware analysis
- Machine learning detection

- Commercial-level antivirus features

10. Future Enhancements

- Real-time file monitoring
- Process monitoring using Windows API
- Strong cryptographic hashing (SHA-256)
- Graphical User Interface (GUI)
- Automatic signature updates

11. Conclusion

This project demonstrates how antivirus mechanisms can integrate with operating system components using C++. It provides practical exposure to file handling, directory management, and system-level programming concepts relevant to operating systems and cybersecurity.

The project strengthens understanding of OS security fundamentals while maintaining simplicity and clarity suitable for academic implementation.